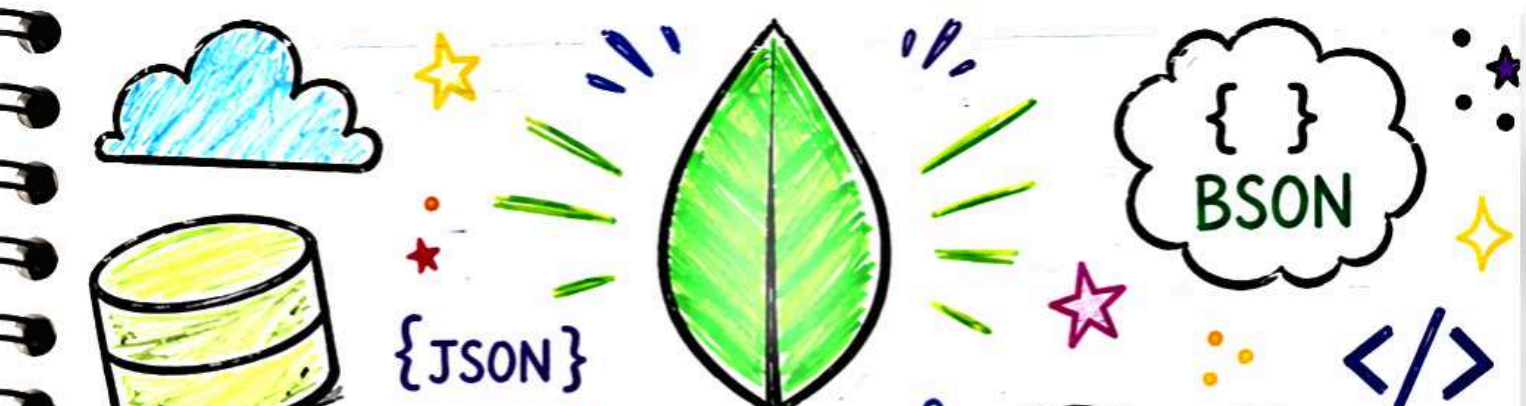




MongoDB

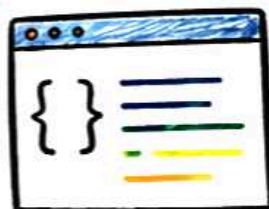
CHEAT SHEET



EVERY

DEVELOPER

NEEDS





MongoDB BASICS



1. WHAT IS MONGODB?



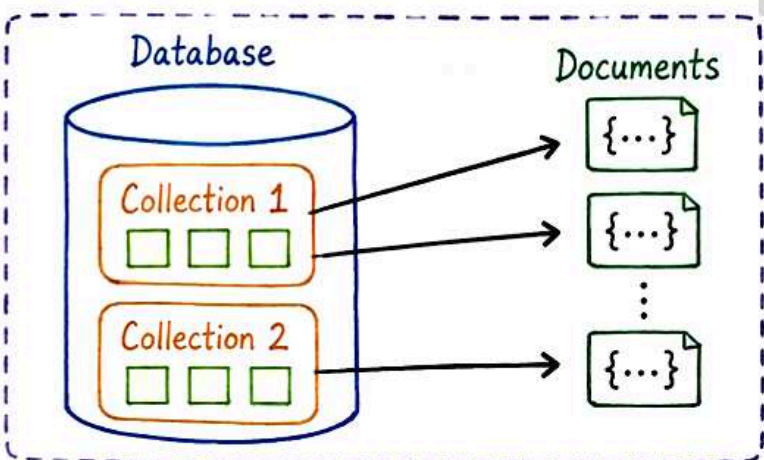
mongoDB

- NoSQL Database
- Stores data in **JSON-like BSON** format
- Flexible schema
- High performance and scalability



2. KEY TERMS

- **Database** → Collection of collections
- **Collection** → Group of documents
- **Document** → Record (similar to row)
- **Field** → Key-value pair

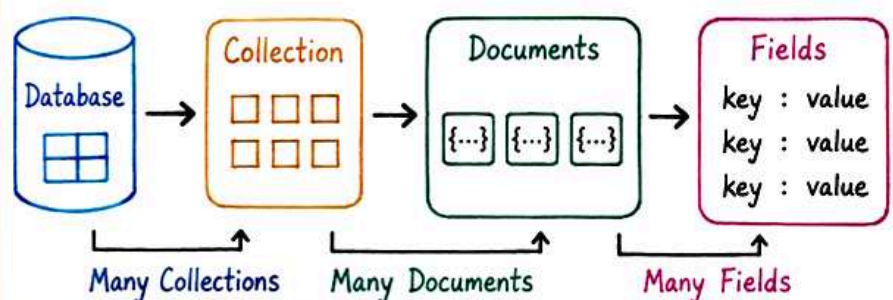


3. EXAMPLE DOCUMENT

```
{
  "name": "Pranjal",
  "age": 22,
  "skill": "MongoDB"
}
```

JSON-like
BSON

4. STRUCTURE FLOW



5. FEATURES

- ✓ Schema-less (Flexible)
- ✓ Easy to Scale
- ✓ High Performance
- ✓ Replication & High Availability
- ✓ Support for Big Data



6. COMMON USE CASES

- ★ Web & Mobile Applications
- ★ Content Management Systems
- ★ Real-time Analytics
- ★ IoT Applications
- ★ Logging & Monitoring

★ MongoDB = Flexible Data Model + High Performance + Easy Scalability





MongoDB

DATABASE COMMANDS



1

SHOW DATABASES

show dbs



Lists all available databases.

EXAMPLE OUTPUT

admin	0.000GB
config	0.000GB
local	0.000GB
coderstar	0.002GB



2

CREATE / USE DATABASE

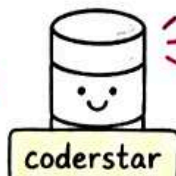
use coderstar



Creates the database if not exists & switches to it.

EXAMPLE

switched to db coderstar



3

CURRENT DATABASE

db



Shows the currently selected database.

EXAMPLE

coderstar



You are here!

4

DROP DATABASE

db.dropDatabase()



Deletes the current selected database.

EXAMPLE

{ "ok": 1, "dropped": "coderstar" }



5

SHOW COLLECTIONS

show collections



Lists all collections in the current database.

EXAMPLE OUTPUT

students
teachers
courses



6

CREATE COLLECTION

db.createCollection("students")



Creates a new collection.

EXAMPLE

{ "ok": 1 }



Think: Database → Collection → Document → Field

KEEP PRACTICING!





MongoDB

INSERT OPERATIONS



We insert data into a collection (**students**)

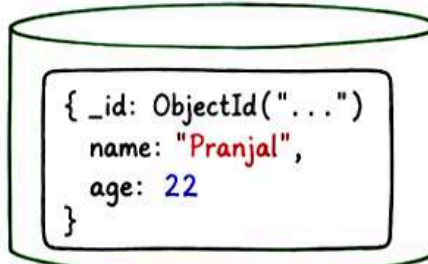


Let's Add Data!

1 INSERT ONE DOCUMENT

```
db.students.insertOne({  
  name: "Pranjal",  
  age: 22  
})
```

Collection: **students**



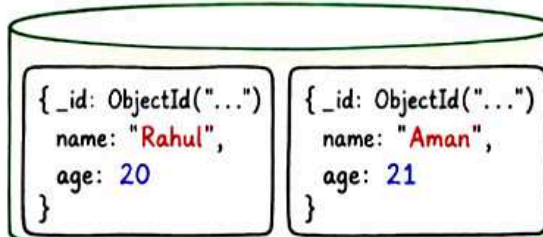
One Document Inserted!



2 INSERT MULTIPLE DOCUMENTS

```
db.students.insertMany([  
  {name: "Rahul", age: 20},  
  {name: "Aman", age: 21}  
])
```

Collection: **students**



Multiple Documents Inserted!

3 VIEW DATA

```
db.students.find()
```



Shows all documents in the collection

Output (Raw)

```
{ "_id" : ObjectId("..."), "name" : "Pranjal", "age" : 22 }  
{ "_id" : ObjectId("..."), "name" : "Rahul", "age" : 20 }  
{ "_id" : ObjectId("..."), "name" : "Aman", "age" : 21 }
```



4 PRETTY FORMAT

```
db.students.find().pretty()
```



More readable and easy to read!

Output (Pretty)

```
{  
  "_id" : ObjectId("..."),  
  "name" : "Pranjal",  
  "age" : 22  
}  
{  
  "_id" : ObjectId("..."),  
  "name" : "Rahul",  
  "age" : 20  
}  
{  
  "_id" : ObjectId("..."),  
  "name" : "Aman",  
  "age" : 21  
}
```

QUICK FLOW



Collection (**students**)



Insert Data



View Data



Pretty Format

REMEMBER

- ✓ insertOne() → one document
- ✓ insertMany() → multiple documents
- ✓ find() → view data
- ✓ find().pretty() → readable view

Practice More, Get Better!



Practice Today, Master Tomorrow!



MongoDB

QUERY OPERATIONS

1 FIND ONE

```
db.students.findOne()
```

OUTPUT

```
{ _id: 1, name: "Pranjal",  
  age: 22, city: "Delhi" }
```

2 FIND BY CONDITION

```
db.students.find({age: 22})
```

OUTPUT

```
{ _id: 1, name: "Pranjal", age: 22, city: "Delhi" }  
{ _id: 4, name: "Aman", age: 22, city: "Pune" }
```

3 GREATER THAN (> 20)

```
db.students.find({age: {$gt: 20}})
```

OUTPUT

```
{ _id: 1, name: "Pranjal", age: 22, city: "Delhi" }  
{ _id: 4, name: "Aman", age: 22, city: "Pune" }  
{ _id: 5, name: "Rohan", age: 24, city: "Noida" }
```

4 LESS THAN (< 25)

```
db.students.find({age: {$lt: 25}})
```

OUTPUT

```
{ _id: 1, name: "Pranjal", age: 22, city: "Delhi" }  
{ _id: 2, name: "Rahul", age: 20, city: "Mumbai" }  
{ _id: 3, name: "Neha", age: 21, city: "Delhi" }  
{ _id: 4, name: "Aman", age: 22, city: "Pune" }
```

5 AND OPERATOR

both conditions

```
db.students.find({  
  age: {$gt: 20},  
  city: "Delhi"  
})
```

OUTPUT

```
{ _id: 1, name: "Pranjal",  
  age: 22, city: "Delhi" }
```

6 OR OPERATOR

any one condition

```
db.students.find({  
  $or: [  
    {city: "Delhi"},  
    {city: "Mumbai"}  
  ]  
})
```

OUTPUT

```
{ _id: 1, name: "Pranjal", age: 22, city: "Delhi" }  
{ _id: 2, name: "Rahul", age: 20, city: "Mumbai" }  
{ _id: 3, name: "Neha", age: 21, city: "Delhi" }
```

7 QUICK REFERENCE

- **\$gt** → Greater Than (>)
- **\$lt** → Less Than (<)
- **\$or** → OR (any one)
- (comma) → AND (all)

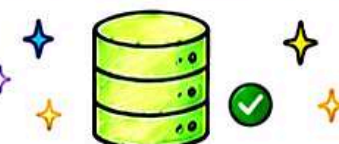
Remember!

- AND → use comma (multiple conditions)
- OR → use **\$or** array

Query =
Find the
RIGHT Data!

SAMPLE COLLECTION (students)

_id	name	age	city
1	Pranjal	22	Delhi
2	Rahul	20	Mumbai
3	Neha	21	Delhi
4	Aman	22	Pune
5	Rohan	24	Noida



TIPS

- ✓ **findOne()** → first match
- ✓ **find({})** → all docs
- ✓ Use conditions to filter data
- ✓ Operators make queries powerful!

OPERATOR LEGEND

- > **\$gt** → Greater Than
- < **\$lt** → Less Than
- = (value) → Equal To
- \$or** → OR (any one)
- , → AND (all)

MongoDB

UPDATE OPERATIONS



We modify existing data in a collection

SAMPLE COLLECTION (students)

_id	name	age	status
1	Pranjal	22	Student
2	Rahul	20	Student
3	Aman	21	Student

1 UPDATE ONE

Update single document

```
db.students.updateOne(
  {name: "Pranjal"},
  {$set: {age: 23}}
)
```

BEFORE

```
{_id: 1, name: "Pranjal",
  age: 22, status: "Student"}
```



AFTER

```
{_id: 1, name: "Pranjal",
  age: 23, status: "Student"}
```

Updated one document!

2 UPDATE MANY

Update multiple documents

```
db.students.updateMany(
  {},
  {$set: {status: "Active"}}
)
```

BEFORE

```
{_id: 1, name: "Pranjal", age: 22, status: "Student"}
{_id: 2, name: "Rahul", age: 20, status: "Student"}
{_id: 3, name: "Aman", age: 21, status: "Student"}
```



AFTER

```
{_id: 1, name: "Pranjal", age: 22, status: "Active"}
{_id: 2, name: "Rahul", age: 20, status: "Active"}
{_id: 3, name: "Aman", age: 21, status: "Active"}
```

Updated many documents!

3 INCREMENT VALUE

Increase numeric value

```
db.students.updateOne(
  {name: "Pranjal"},
  {$inc: {age: 1}}
)
```

BEFORE

```
{_id: 1, name: "Pranjal", age: 23, status: "Active"}
```



AFTER

```
{_id: 1, name: "Pranjal", age: 24, status: "Active"}
```

Increased value by 1!

4 RENAME FIELD

Rename a field

```
db.students.updateMany(
  {},
  {$rename: {"status": "userStatus"}}
)
```

BEFORE

```
{_id: 1, name: "Pranjal", age: 24, status: "Active"}
{_id: 2, name: "Rahul", age: 20, status: "Active"}
{_id: 3, name: "Aman", age: 21, status: "Active"}
```



AFTER

```
{_id: 1, name: "Pranjal", age: 24, userStatus: "Active"}
{_id: 2, name: "Rahul", age: 20, userStatus: "Active"}
{_id: 3, name: "Aman", age: 21, userStatus: "Active"}
```

Field renamed successfully!

UPDATE OPERATORS USED

- ★ \$set → Set new value
- ★ \$inc → Increment number
- ★ \$rename → Rename field
- ★ {} → Empty filter = all documents

QUICK FLOW

Find → Update → Result



REMEMBER!

- ✓ updateOne() → one document
- ✓ updateMany() → many documents
- ✓ Use conditions {} to target specific docs



Practice More ★ Write Code → Master MongoDB



MongoDB

DELETE OPERATIONS



We remove data from a collection using **delete** operations.

Keep your data clean!

SAMPLE COLLECTION (students)

_id	name	age	city
1	Pranjal	22	Delhi
2	Rahul	20	Mumbai
3	Aman	17	Pune
4	Neha	19	Delhi
5	Rohan	16	Noida

1 DELETE ONE DOCUMENT

```
db.students.deleteOne({
  name: "Pranjal"
})
```

BEFORE

_id	name	age	city
1	Pranjal	22	Delhi
2	Rahul	20	Mumbai
3	Aman	17	Pune
4	Neha	19	Delhi
5	Rohan	16	Noida

AFTER

_id	name	age	city
2	Rahul	20	Mumbai
3	Aman	17	Pune
4	Neha	19	Delhi
5	Rohan	16	Noida

Only one matching document is deleted!

2 DELETE MULTIPLE DOCUMENTS

```
db.students.deleteMany({
  age: { $lt: 18 }
})
```

BEFORE

_id	name	age	city
1	Pranjal	22	Delhi
2	Rahul	20	Mumbai
3	Aman	17	Pune
4	Neha	19	Delhi
5	Rohan	16	Noida

AFTER

_id	name	age	city
1	Pranjal	22	Delhi
2	Rahul	20	Mumbai
4	Neha	19	Delhi

All docs with age less than 18 are deleted!

3 REMOVE ALL DOCUMENTS

```
db.students.deleteMany({})
```

BEFORE

_id	name	age	city
1	Pranjal	22	Delhi
2	Rahul	20	Mumbai
3	Aman	17	Pune
4	Neha	19	Delhi
5	Rohan	16	Noida

AFTER

(Empty Collection)

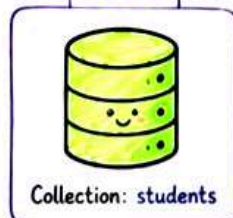


All documents are removed but collection still exists!

4 DROP COLLECTION

```
db.students.drop()
```

BEFORE



AFTER

Collection Deleted!

Collection: students
(Does not exist)

Collection and all data are permanently removed!



BE CAREFUL!

- ✓ Deleted data cannot be undone.
- ✓ Always use conditions {} while deleting.

QUICK FLOW



SUMMARY

- ✓ deleteOne() → one document
- ✓ deleteMany() → many / all documents
- ✓ drop() → drop collection

★ Practice More → Write Code → Master MongoDB



MongoDB



USEFUL COMMANDS



Short, simple and super useful commands you'll use everyday! 😊

Sample Collection: students

_id	name	age	city
1	Pranjal	22	Delhi
2	Rahul	20	Mumbai
3	Aman	17	Pune
4	Neha	19	Delhi
5	Rohan	16	Noida
6	Kavya	23	Bangalore

1 COUNT DOCUMENTS

```
db.students.countDocuments()
```

OUTPUT

6

(Total documents)



2 SORT DATA (ASCENDING)

```
db.students.find().sort({age: 1})
```

OUTPUT (age: 1)

[16, 17, 19, 20, 22, 23]

(Smallest → Largest)



★ DESCENDING ORDER

```
db.students.find().sort({age: -1})
```

OUTPUT (age: -1)

[23, 22, 20, 19, 17, 16]

(Largest → Smallest)



3 LIMIT RESULTS

```
db.students.find().limit(5)
```

OUTPUT (First 5 docs)

[1, 2, 3, 4, 5]

(Only 5 documents)



4 SKIP RESULTS

```
db.students.find().skip(5)
```

OUTPUT (After skipping 5)

[6]

(Starts from 6th doc)



5 CREATE INDEX

```
db.students.createIndex({name: 1})
```

OUTPUT

Index created successfully



{ field: 1 } → Ascending index | { field: -1 } → Descending index

6 MONGODB CONNECTION

```
mongodb://localhost:27017
```

OUTPUT

Connected to MongoDB Server



7 MONGOOSE CONNECTION

```
mongoose.connect('mongodb://localhost:27017/mydb')
```

OUTPUT

Connected to mydb database



REMEMBER

- 1 → Ascending Order
A to Z / 0 to 9
- 1 → Descending Order
Z to A / 9 to 0

INDEX = SPEED

Indexes make search operations faster!



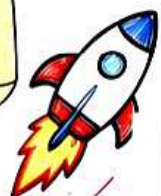
TIPS

- ✓ Use indexes for fields you search often.
- ✓ Skip + Limit = helpful for pagination.
- ✓ Sort, Limit, Skip are chainable!

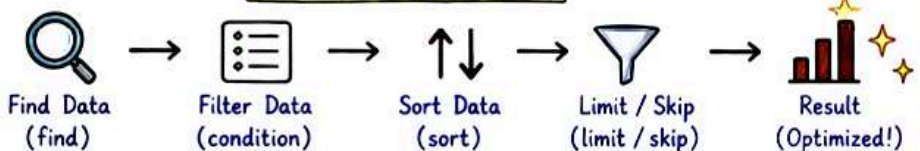


NOTE TO SELF

Practice these commands daily and become a MongoDB Pro! 🌟😎



QUICK COMMAND FLOW



Practice More → Write Code → Build Projects → Get Better

